

ASL-SR-DPT CO-AUTHOR BEGINNER TESTING MANUAL

Exact operational instructions for co-authors who do not know the paper, Python project, solver terminology, or benchmark workflow

Goal: Follow this manual from top to bottom. Do not improvise benchmark settings. When a step says to edit a value, use the exact value shown for the approved experiment. Keep the raw results.

The FSUU Institutional Student Research Manual specifies Chapter 3 as **RESULTS AND ANALYSES** and directs researchers to present findings according to the sequence of the research problems or objectives in Chapter 1. Methodology belongs in Chapter 2.

1. What You Are Testing

The research evaluates the proposed **Accelerated Single-Loop Sparse Recovery with Dynamic Parameter Tuning (ASL-SR-DPT)** solver for compressive image denoising on the BSD68 benchmark.

Component	Use in the experiment
Dataset	Complete BSD68 grayscale benchmark, 68 images
Noise	AWGN with $\sigma_{\text{noise}} = 15, 25, 50$
Patch	8 x 8 pixels
DCT coefficients	$N = 64$
Measurements	$M = 38$
Measurement ratio	$M/N = 0.6$
Sensing modes	Standard random, DC-preserving
Solvers	ASL-SR-DPT, OMP, LASSO-ADMM
Patch stride	2 pixels
Threads	1 numerical thread
Randomized repetitions	50 trials per defined condition

The Chapter 3 guide uses the same experiment groups and requires raw results to be saved before summary statistics are calculated. [filecite turn28file0 L259-L293](#)

2. Words You Need to Know

Word	Simple meaning
BSD68	Fixed set of 68 natural grayscale benchmark images used for final evaluation.
Patch	An 8 x 8 part of an image.
DCT	A transform that changes an image patch into 64 frequency coefficients.
Sensing matrix A	A 38 x 64 matrix used to compress 64 DCT coefficients into 38 measurements.
Measurement y	The compressed values supplied to a solver.
Solver	An algorithm that tries to recover the 64 coefficients from the compressed measurements.
Baseline	A comparison solver, OMP or LASSO-ADMM.
Trial	One randomized repetition under one defined experimental condition.
Seed	A number controlling a reproducible random sequence.

Word	Simple meaning
PSNR	Image-fidelity score. Higher is better.
SSIM	Structural-similarity score. Higher is better.
MSE	Pixel error. Lower is better.
Solve time	Time spent executing the solver under the defined timing boundary.
Active support	Coefficient positions currently included in the ASL-SR-DPT gradient calculation.
Raw data	Original per-run measurements. Never overwrite them.

3. Before You Touch the Code

Do not start the full benchmark until every item below is complete.

- 1 Create a backup of the complete project folder.
- 2 Create a separate frozen copy of the final solver.
- 3 Record the current code version or commit ID.
- 4 Verify the BSD68 files.
- 5 Verify the VM configuration.
- 6 Verify Python and numerical-library versions.
- 7 Verify the CSV output code.
- 8 Run the smoke test.

If a bug appears during final data collection, stop. Create a new code version, document the change, and rerun the affected condition. Do not edit old raw results.

4. Start VMware and Open Ubuntu

- 1 Open VMware Workstation Pro on the Windows host.
- 2 Start the Ubuntu 24.04 LTS virtual machine used for this research.
- 3 Log into Ubuntu.
- 4 Open Terminal.

The documented VM uses 4 vCPUs, 8 GB RAM, SSD-backed storage, disabled 3D acceleration, and a single-thread numerical backend. [filecite turn24file0 L188-L205](#)

5. Verify Python and Libraries

```
python3 --version python3 -c "import numpy, scipy; print('NumPy', numpy.__version__); print('SciPy', scipy.__version__)"
```

The current methodology documents Python 3.12.3, NumPy 1.26.4, and SciPy 1.12.0. If the installed versions differ, stop and tell the lead author before benchmarking. [filecite turn24file0 L210-L214](#)

6. Lock Numerical Threading

Paste these commands into Terminal:

```
export OMP_NUM_THREADS=1 export OPENBLAS_NUM_THREADS=1 export MKL_NUM_THREADS=1 export NUMEXPR_NUM_THREADS=1
```

Check:

```
echo $OMP_NUM_THREADS $OPENBLAS_NUM_THREADS $MKL_NUM_THREADS $NUMEXPR_NUM_THREADS
```

Expected output: **1 1 1 1**.

7. Find the Project Files

Open the research project in your code editor. Typical files may include:

```
ASL-SR-DPT.py benchmark.py sensing.py metrics.py reconstruction.py streamlit_app.py
```

Your actual filenames may differ. Use the files in your project rather than renaming them.

7.1 Find the solver

- 1 Open the file containing the HybridSparseSolver or ASL-SR-DPT solver class.
- 2 Press Ctrl+F.
- 3 Search for **lambda_reg**.
- 4 Search for **sigma_min**.
- 5 Search for **decrease_factor**.
- 6 Search for **cost_mid**.
- 7 Search for **active_indices**.

8. Record the Frozen Base Parameters

The following values are documented in your current research materials. Confirm them against the actual V7 code before running final tests.

Parameter	Value to verify
lambda_reg	Current documented default: 0.1
tol	1e-5
sigma_min	0.01
sigma decay	0.95
support threshold	1e-5 x sigma
support reopening	Every 3 iterations
Armijo c	1e-4
beta_decay	0.5
Initial mu	0.1 in the earlier supplied solver. Confirm V7.
max_iter	Confirm the actual V7 value.

Do not change a value simply to make the result better. Parameter changes belong in the documented validation experiments.

9. Create final_config.txt

Create a text file inside the benchmark folder. Copy this template and fill in the actual frozen values:

```
patch_size=8 stride=2 N=64 M=38 compression_ratio=0.6 noise_levels=15,25,50 sigma_min=0.01 sigma_decay=0.95
support_threshold_multiplier=1e-5 support_reopen_interval=3 armijo_c=1e-4 beta_decay=0.5 threads=1
lambda_reg=FILL_ACTUAL_FROZEN_VALUE max_iter=FILL_ACTUAL_FROZEN_VALUE
```

Save this file with every final benchmark package.

10. Smoke Test

Purpose: make sure the complete pipeline works before large runs.

- 1 Choose one BSD68 image.
- 2 Choose noise = 15.
- 3 Choose standard random sensing.
- 4 Process one 8 x 8 patch.
- 5 Run 2D DCT.
- 6 Generate A.
- 7 Generate y.
- 8 Run ASL-SR-DPT.
- 9 Run OMP with the same A and y.
- 10 Run LASSO-ADMM with the same A and y.
- 11 Run IDCT on recovered coefficients.
- 12 Compute PSNR, SSIM, and MSE.
- 13 Check the console and output files for errors.

Expected dimensions: patch = 8 x 8, DCT = 64 values, A = 38 x 64, A-dagger = 64 x 38, y = 38 values, recovered z = 64 values.

11. DCT -> IDCT Test

- 1 Select any clean 8 x 8 patch.
- 2 Run DCT.
- 3 Immediately run IDCT without sensing or denoising.
- 4 Compare the output to the original patch.
- 5 Record the reconstruction error.

If DCT followed by IDCT does not recover the original patch to numerical precision, stop. Fix the transform implementation before any solver experiment.

12. Sensing Dimension Test

- 1 Generate the standard sensing matrix.
- 2 Print its shape.
- 3 Confirm A is (38, 64).
- 4 Compute the pseudoinverse.
- 5 Confirm A-dagger is (64, 38).
- 6 Generate y.
- 7 Confirm y has length 38.

Do not continue if the dimensions do not match.

13. Gradient Check

The Chapter 3 guide recommends a finite-difference check as evidence of implementation correctness.

filecite[turn28file0][L81-L85]

```
Relative Gradient Error = ||g_analytic - g_numeric||_2 / max(1, ||g_numeric||_2)
```

- 1 Choose a fixed test z .
- 2 Compute the analytical gradient from ASL-SR-DPT.
- 3 Compute a finite-difference numerical gradient.
- 4 Calculate the relative error.
- 5 Save the result and test conditions.

14. Experiment E1 - Solver Verification

- 1 Run the dimension tests.
- 2 Run the DCT/IDCT test.
- 3 Run the gradient test.
- 4 Run one ASL-SR-DPT patch.
- 5 Record objective, final sigma, iterations, residual, and line-search failures.
- 6 Confirm no NaN or Inf.

Save E1 separately from final benchmark data.

15. Experiment E4 - Midpoint ON vs OFF

This experiment requires two solver copies.

15.1 Create copies

- 1 Copy the frozen solver file.
- 2 Name the first copy ASL-SR-DPT_midpoint_on.py.
- 3 Name the second copy ASL-SR-DPT_midpoint_off.py.

15.2 Find the midpoint block

- 1 Open the solver copy.
- 2 Press Ctrl+F.
- 3 Search for **cost_mid**.
- 4 Search for **z_mid**.
- 5 Find the conditional branch accepting the midpoint candidate.

15.3 Midpoint ON

Leave the existing midpoint acceptance branch unchanged. Use this copy for the enabled condition.

15.4 Midpoint OFF

Disable only the midpoint acceptance branch in the OFF copy. Keep the main Armijo candidate test unchanged.

Run identical validation conditions for ON and OFF. Record PSNR, SSIM, MSE, runtime, iterations, and failed line searches. The Chapter 3 guide explicitly says not to assume the midpoint is beneficial before measuring it.

■filecite■turn28file0■L162-L165■

16. Experiment E5 - Sigma Schedule

The documented baseline schedule is:

```
sigma = max(sigma * 0.95, 0.01)
```

For validation, use only the predefined schedules approved by the lead author. Example schedules such as 0.90, 0.95, and 0.98 are suggestions for a validation grid, not final study values until approved.

How to edit:

- 1 Make a copy of the solver for the schedule being tested.
- 2 Press Ctrl+F and search for **decrease_factor** or **0.95**.
- 3 Change only the schedule value.
- 4 Save with the value in the filename.
- 5 Run the validation data.
- 6 Restore the frozen baseline after the comparison.

17. Experiment E6 - Lambda Sensitivity

The previously supplied solver uses **lambda_reg=0.1** as the documented default. Confirm the V7 value before testing.

- 1 Open the solver.
- 2 Press Ctrl+F and search for **lambda_reg**.

- 3 Record the current value.
- 4 Create a separate validation copy for each approved lambda.
- 5 Run each lambda under identical validation conditions.
- 6 Record PSNR, SSIM, MSE, iterations, runtime, active-support ratio, and failed line searches.
- 7 Select the final lambda using the validation rule.
- 8 Freeze the selected lambda.

Do not select lambda using final BSD68 test performance. The Chapter 3 guide requires parameter selection on validation data before final evaluation. ■filecite■turn28file0■L166-L172■

18. Freeze the Code and Parameters

- 1 Choose the final validated lambda.
- 2 Choose the final validated sigma schedule.
- 3 Choose the final midpoint setting according to the study design.
- 4 Record the final values in final_config.txt.
- 5 Save the frozen solver under a versioned name.
- 6 Do not edit the frozen solver during the final benchmark.
- 7 Record the commit ID or file hash.

From this point onward, final benchmark runs must use the frozen code.

19. Final Benchmark E2 - Standard Random Sensing

Do this after the parameters are frozen.

- 1 Set sensing mode = standard random.
- 2 Run noise = 15.
- 3 Process all 68 BSD68 images.
- 4 Repeat according to the defined 50-trial protocol.
- 5 Run noise = 25 under the same protocol.
- 6 Run noise = 50 under the same protocol.
- 7 Save raw results after every completed trial.

20. Final Benchmark E3 - DC-Preserving Sensing

Run the same final benchmark framework with the DC-preserving sensing configuration.

- 1 Use the same 68 BSD68 images.
- 2 Use the same randomized trial structure.
- 3 Retain the observed noisy DC coefficient.
- 4 Compress only the 63 AC coefficients.
- 5 Reassemble the 64-coefficient vector.
- 6 Run IDCT and normalized Hamming aggregation.
- 7 Save PSNR, SSIM, MSE, runtime, and solver diagnostics.

The preserved DC coefficient remains noisy. This experiment isolates the effect of avoiding compression of the DC coefficient.

■filecite■turn24file0■L263-L272■

21. Trial Fairness Rules

For every matched ASL-SR-DPT, OMP, and LASSO-ADMM comparison, use:

- same clean image
- same noisy image
- same sensing matrix A
- same measurement vector y

- same patch coordinates
- same reconstruction pipeline
- same thread configuration
- same timing boundary

This prevents the algorithms from being tested on different inverse problems. The existing Chapter 3 guide explicitly requires matched A and y. ■filecite■turn28file0■L276-L282■

22. Run OMP

- 1 Use the same A and y.
- 2 Use the frozen column-normalized projection implementation.
- 3 Use the frozen residual stopping rule.
- 4 Save the recovered coefficient vector.
- 5 Measure solve time using the same timing boundary.

23. Run LASSO-ADMM

- 1 Use the same A and y.
- 2 Use the frozen lambda_lasso and rho values.
- 3 Use the frozen Cholesky implementation.
- 4 Use primal and dual residual stopping.
- 5 Save the recovered coefficient vector.
- 6 Measure solve time using the same timing boundary.

24. Reconstruct the Full Image

- 1 Convert recovered coefficient vectors into 8 x 8 DCT coefficient arrays.
- 2 Apply 2D IDCT to each patch.
- 3 Place each patch at its original location.
- 4 Apply the normalized 2D Hamming weights.
- 5 Accumulate weighted pixels.
- 6 Accumulate Hamming weights.
- 7 Divide accumulated pixel values by accumulated weights.
- 8 Save the final reconstructed spatial image.

Use exactly the same aggregation implementation for all methods.

25. Compute the Metrics

Metric	Record
Noisy PSNR	Noisy image versus clean reference
PSNR	Final reconstructed image versus clean reference
SSIM	Final reconstructed image versus clean reference
MSE	Final reconstructed image versus clean reference
Solve time	Iterative solver time
Setup time	One-time setup, reported separately
Iterations	Actual solver iterations
Final sigma	Final continuation parameter
Residual	Final solver residual
Active-support ratio	Average active coefficient utilization
Failed line searches	Count of failures

```
PSNR = 10 log10(MAX2 / MSE)
```

```
R_active = [sum from i=1 to k_max of ||z_active(i)||0] / (k_max x N)
```

For the 8 x 8 DCT patch, N = 64. Use the actual executed iteration count for k_max. ■filecite■turn28file0■L106-L111■

26. Save Raw Data

Recommended CSV header:

```
trial,image_id,noise_sigma,sensing_mode,solver,psnr,ssim,mse,solve_time,setup_time,iterations,final_sigma,residual,active_support_ratio,failed_line_searches
```

Add these fields when available: run_id, code_version, seed, image_width, image_height, patch_count, vm_label.

Do not overwrite raw CSV files. Generate a new file or run folder for a new benchmark execution.

27. Audit Every Row

- No NaN or Inf.
- PSNR, SSIM, and MSE are present.
- Solve time is positive.

- Iterations are present.
- Image ID, trial, noise, sensing, and solver are present.
- No duplicate observation keys.
- Matched methods received the same A and y.
- Reconstruction files exist for selected visual examples.

Do not silently delete a failed observation. Create a failed-run log and rerun the condition after the cause is corrected.

28. What the Co-Author Should Run, in Order

Order	Action
1	Start VMware and Ubuntu
2	Verify Python/NumPy/SciPy
3	Set four numerical thread variables to 1
4	Verify BSD68: all 68 images
5	Run DCT -> IDCT test
6	Run dimension test
7	Run numerical gradient check
8	Run smoke test
9	Run midpoint validation
10	Run sigma schedule validation
11	Run lambda validation
12	Freeze final code and parameters
13	Run E2 standard sensing
14	Run E3 DC-preserving sensing
15	Run E7 ASL-SR-DPT vs OMP
16	Run E8 ASL-SR-DPT vs LASSO-ADMM
17	Audit raw CSV
18	Generate summaries and figures
19	Hand evidence package to lead author

29. Do Not Change These During Final Runs

- Patch size
- Patch stride
- Noise level selected for the current condition
- Sensing matrix definition
- Solver equations
- lambda
- sigma schedule
- midpoint setting
- stopping rules
- thread configuration
- timing boundary
- image aggregation method

Any deliberate change creates a new experiment or new code version and must be recorded.

30. Required Chapter 3 Evidence

Chapter 3 section	Evidence
3.1 Mathematical verification	Dimensions, objective, gradient check, finite values
3.2 Armijo/continuation	Objective, mu, sigma, accepted/failed steps
3.3 Active support	R_active, support counts, reopening behavior
3.4 SVD initialization	A-dagger versus controlled initialization
3.5 Spatial reconstruction	Images, PSNR, SSIM, MSE
3.6 Ablations	Sensing, midpoint, sigma, lambda
3.7 OMP comparison	Matched quality/runtime results and speedup
3.8 LASSO comparison	Matched quality/runtime results and speedup
3.9 Overall trade-off	Quality/runtime/support summary
3.10 Prototype	Streamlit screenshots and outputs
3.11 Objective map	Each Chapter 1 objective linked to evidence

31. When Results Look Bad

- 1 Check code version.
- 2 Check DCT and IDCT.
- 3 Check A dimensions.
- 4 Check measurement vector y.
- 5 Check objective and gradient.
- 6 Check Armijo condition.
- 7 Check sigma schedule.
- 8 Check lambda.
- 9 Check DC-preserving handling.
- 10 Check Hamming aggregation.
- 11 Check PSNR, SSIM, and MSE calculations.
- 12 Rerun the condition.

If the poor result remains valid after verification, keep it. Do not change the research or alter valid measurements to improve the result. The existing Chapter 3 guide explicitly treats inferior results as legitimate findings when implementation and experiment are valid. ■filecite■turn28file0■L196-L213■

32. Raw Data and Handover Checklist

- All raw CSV files
- All summary CSV files
- Parameter-validation outputs
- Objective/mu/sigma plots
- Active-support plots
- Reconstructed image panels
- OMP comparison
- LASSO-ADMM comparison
- Midpoint comparison
- Sigma comparison
- Lambda comparison
- VM configuration record
- Code version/commit ID
- Failed-run log

Do not send only screenshots. Raw numerical data and metadata are the primary evidence.

33. Final 50-Trial Operator Checklist

- 1 Record trial number.
- 2 Record seed.
- 3 Record noise level.
- 4 Record sensing mode.
- 5 Use frozen code.
- 6 Use frozen parameters.
- 7 Process the 68 BSD68 images.
- 8 Use matched measurements for the comparison solvers.
- 9 Save raw observations.
- 10 Audit the raw observations.
- 11 Do not overwrite previous trials.

34. Final Benchmark Checklist

Requirement	Complete
All 68 BSD68 images included	■
Noise 15, 25, 50 included	■
Standard random sensing completed	■
DC-preserving sensing completed	■
Midpoint ablation completed	■

Requirement	Complete
Sigma schedule validation completed	■
Lambda validation completed	■
OMP comparison completed	■
LASSO-ADMM comparison completed	■
Raw CSV preserved	■
Setup and solve times separated	■
PSNR/SSIM/MSE recorded	■
Active-support ratio recorded	■
Matched y/A used for comparisons	■
Final parameters frozen before final testing	■

35. Final Workflow

Chapter 1 objectives -> Chapter 2 methodology -> code freeze -> smoke test -> validation experiments -> parameter freeze -> final benchmark -> raw CSV -> automated summaries -> figures -> Chapter 3 Results and Analyses -> Chapter 4 Summary, Conclusions and Recommendations

This manual expands the uploaded Chapter 3 guide into direct operating instructions for co-authors. The guide defines the E1-E8 experimental matrix, fairness rules, raw CSV schema, and evidence requirements.

■filecite■turn28file0■L259-L321■